Navendran Naidoo
u21512494

University of Pretoria ■ EBIT Faculty ■ **Department of Computer Science** ■ Academic Year **2024**

# COS341 **Compiler** Construction ■ *Final Exam*

Monday the **11th of November 2024**, Hall "Centenary 1", **11:15h–14:15h**

**This question-booklet must be filled in and returned together with the answer-booklet**



**Internal Examiner:** Prof. *Stefan Gruner*
**External Examiner:** Prof. *Bruce Watson*, University of Kwa Zulu Natal (UKZN)

**INSTRUCTIONS:**
**Read** each task carefully before writing your answers *neatly* into the answer-booklet provided. Please **avoid** (as far as possible) to fill the answer-booklet with irrelevant rough-scribbles: instead, use your own rough-scribble pad for such purposes.
**Wait** *until* the invigilators give the *start-signal* before you begin to work.
Write with **indelible ink**.
*You may solve the given tasks in any sequence that you prefer:*
*Begin with those tasks that you find most easy,* and proceed from there to the more difficult tasks.

**FORBIDDEN**:
- Any kind of *academic dishonesty* (as defined by the rule book of the University of Pretoria);
- Any *electronic or computational devices* such as portable computers, "smart" cell-phones, "smart" wrist-watches, e-book display devices (e.g.: "Kindle"), and the like;
- The *textbook as a whole*;
- *Prints* from 3rd-party materials which you have not authored, including the E-book version of our textbook, photos of the lecture slides, internet wikipedia web-pages, scientific papers from GoogleScholar, etc.;
- Writing with *pencil that can be erased* by a rubber-gum.

**ALLOWED**:
- Student's own self-made (*hand-written*) crib-notes booklet, firmly bound with a solid spine (no "flying" pages), such as shown (for example) in the COS341 Study-Guide on page 4.

**As per COS341 Study-Guide, this Exam has a total Assessment Value of 60 Points**

Task A: ..... [7 Points]
Task B: ..... [14 Points]
Task C: ..... [15 Points]
Task D: ..... [13 Points]
Task E: ..... [11 Points]

**Timing Advice:**
As there are 5 Tasks for 3 hours of work-time, the *average work-time per task* is *approximately half an hour.* However, *some of the tasks can be done much faster*, such that you will have ample time available for the more "tedious" tasks.

CamScanner

**Task A.** [7P]

Given is this following *regular expression* (RE) which describes the structure of ***dual numbers***: besides the neutral number ***0***, some of them are positive, others are negative, some of them are whole numbers, others are fractions. In this scenario, the Afrikaans ***comma*** (instead of the English dot) is used for the syntax of the fraction numbers. Thus the given regular expression's *alphabet* is: **Σ**(RE) := { ***0*** ; ***1*** ; **–** ; **,** }

with RE :=

| |
|---|
| ***0*** \| ***0,(0\|1)*1*** \| ***– 0,(0\|1)*1*** \| ***1(0\|1)**** \| ***–1(0\|1)**** \| ***1(0\|1)*,(0\|1)*1*** \| ***–1(0\|1)*,(0\|1)*1*** |

***Construct*** the *unique* **minimized Deterministic Finite Automaton** that *accepts* not more and not less than exactly the language ***L***(RE).

- **Note**: ONLY the minimized DFA as the final result will be marked. Any *"rough scribbles"*, which you might perhaps write down "on the way" towards the final result, will *not* be marked.

---

From Chapter #5 and Chapter #6 of our textbook you had studied how to define recursive Type Checker functions as well as recursive Translator functions for loop structures such as the WHILE-DO construct as well as the REPEAT-UNTIL construct. The *well-known* **for-loop**, however, was *not* included in the examples of Chapters #5 and #6 of our book. With this example we want to work in the next two tasks.

*Given* is now the following (incomplete) 'snippet' of some (larger) context-free grammar, *G* :

```
LOOP → for ( INIT , COND , INCR ) { ALGO }
INIT → VAR := number // comment: number is a token from the lexer
INCR → VAR := VAR + number
```

**Task B.** ..... [14P]

*Assume* now that the following two *auxiliary functions* are ***already given*** such that they can be used without needing to know their inner structure:

- boolean ***typeCheck***(ALGO, symbolTable) *// comment: also a symbol table is assumed to exist*
- boolean ***typeCheck***(COND, symbolTable)

On the basis of these "givens", **write a type checking function** of the form

boolean ***typeCheck***(LOOP, symbolTable) { **...** }

*including also* appropriate sub-functions for the type-checking of INIT and of INCR, such that your function

- returns **true** *if* some given **for**-loop-program (within the language of *G*) was *correctly typed*
- returns **false** *otherwise*

You shall present your solution in a "style" that is *similar* to the "style" of Chapter #5, but *keep in mind* that the type checker functions of Chapter #5 are *void* functions (which only print messages but do *not* return anything), whereas *our* type checker functions here are *not printing* any messages but *return* a Boolean *"flag"* instead.

- **ONLY** your final result –*not* the *"rough scribbles"*– will be marked.

CamScanner

**Task C.** [15P]

Also in this task *we still work with the same grammar G* of above.

**Advice:**

VAR := **number** *is very similar to "**id := Atom**"* of *Grammar 6.1* in Textbook Section 6.2
VAR := VAR + **number** *is similar to "**id := id binop Atom**"* of *Grammar 6.1*, Section 6.2

Assume now again that the following *auxiliary functions* are ***already given*** and can be used without needing to know their inner structure:

- ***transStat***(ALGO, symbolTable) *// as in Fig.6.5 of our Textbook*
- ***transStat***( **(id** := EXPR), symbolTable) *// as in Fig.6.5 of our Textbook*
- ***newlabel*****()** *// as in Fig.6.5 of our Textbook*
- ***transCond***(COND, Lt, Lf, symbolTable) *// as in Fig.6.6 of our Textbook*

On the basis of these "givens", **write an intermediate-code generator function** of the form

| ***transStat***(LOOP, symbolTable) { **...** } |
|---|

which *correctly* produces *intermediate code* for any given **for**-loop-program (in the language of *G*).

Thereby you shall present your solution in a "style" that is *very similar* to the "style" of Chapter #6.

- Additional Advice: *Notice* how both INIT as well as INCR of *G match* the above-mentioned *pattern* (**id** := EXPR) which the already given ***transStat***-Function can "consume" as its input
- **ONLY** your final result *–not* the *"rough scribbles"*– will be marked.

**Task D.** [13P]

Given are the following two *symbol tables* which map user-defined names to their internal names.

For **variables**:

| *user-defined name* | *internal new name* |
|---|---|
| **i** | **v0** |

For **functions** and **operations**:

| *user-defined name* | *internal new name* |
|---|---|
| **chainSum** | **f** |
| := | := |
| < | < |
| + | + |
| – | – |

Given is now also the following **source code** which is a *recursive function definition:*

```
chainSum( i )
{
 if ( i < 0 )
      then { i := 0 }
      else { i := i + chainSum( i–1 ) }
 RETURN i
}
```

Because the given function is *recursive*, translation by *inlining* is *not* possible. Therefore, **translate** this function by *accurately* applying the translation methods from textbook Chapter **#6** and Section **#9.3** (*"Caller Saves"*), whereby you shall additionally assume the following two conventions: →

CamScanner

may only be opened by the examiner if the student number or other information is incorrect or illegit

- Upon its first invocation, the auxiliary function **newvar()** will yield **t0** (*as in* Chapter #6), and thereafter **t1**, **t2**, **t3**, etc...
- Upon its first invocation, the auxiliary function **newlabel()** will yield **L0**, and thereafter **L1**, **L2**, **L3**, etc...

Note: your translation must follow the correct "flow" of the recursive translation-calls (as outlined in the book's translation tables) "top-down" through the given program's Syntax Tree; however ***only*** your ***finally*** **generated code** will be marked.

**Task E.** _____ [11P]

Let Q be a proper quadrangle with its four outer edges ***a,b,c,d*** and its two inner diagonals ***x,y***. Since times of antiquity it is known that *for all* such quadrangles:

$$(ac + bd) \geq xy$$

On the basis of this ancient knowledge the following low-level target-code program *checks whether* six positive input numbers (provided by the user) *constitute* a proper quadrangle (whereby for the sake of simplicity we omit the tedious procedure of first verifying that all user-inputs are indeed >0)

```
Line   Command
  1    r := 0          // comment: result = false: no quadrangle
  2    u := a*c
  3    v := b*d
  4    w := u+v
  5    z := x*y
  6    IF w < z THEN e
  7    r := 1          // comment: result = true: quadrangle
  8    LABEL  e
  9    RETURN r
```

With CPU register allocation in mind, provide the ***minimally coloured*** **Interference Graph** for this program.

Additional advice:

- **ONLY** the *minimally coloured Interference Graph* as the final result will be marked!
- In order to "arrive at" this final result you will presumably carry out some *Liveness Analysis* (as in Textbook Chapter #8) as "rough scribbles". These "rough scribbles", however, will *not* be marked.
- ***If* you brought only 1 pen** into this exam, then you can *indicate the colours of the graph's nodes by means of written words* such as "GREEN", "RED", and the like.

---

**- End of Exam Paper: there are *no further questions* -**



CamScanner